

Amplify依存調査と開発方針検討レポート

作成日: 2025年7月5日

作成者: Claude Code

対象プロジェクト: Agile Studio Talent Management System

📄 エグゼクティブサマリー

プロジェクトの引き継ぎ過程で、AWS Amplifyへの依存箇所を包括的に調査しました。当初はモック認証の実装により局所的な問題として対処していましたが、システム全体で約30ファイルにわたる広範囲なAmplify依存が判明しました。本レポートでは現状分析と今後の開発方針について提言します。

重要な発見:

- Amplify依存箇所: 約30ファイル（高影響8件、中影響15件、低影響7件）
- 認証、データ操作、型定義が密結合した構成
- 現在のモック対応は部分的（認証のみ）で、データ操作は未対応

推奨方針: 全面モック化よりもAmplify環境構築を推奨（詳細は後述）

🕒 経緯と現状

初期状況

プロジェクトを引き継いだ際、前開発者がAWS Amplify環境を使用して開発していたことが判明しました。新しい開発環境では以下の課題がありました：

- AWS環境未構築:** 引き継ぎ時点でAmplifyバックエンド環境が利用できない状態
- 認証エラー:** ログイン画面でAmplify認証エラーが発生
- 開発環境構築の緊急性:** 開発を継続するため、迅速な環境構築が必要

対応経緯

Phase 1: モック認証の実装（実施済み）

期間: 2025年7月5日

対応内容:

- モック認証システム（`amplify-mock.ts`）の実装
- 環境変数`VITE_USE MOCK_AUTH`による切り替え機能
- 認証フローのSSR対応（Cookie + localStorage）
- Playwright E2Eテストでの動作確認

結果:

- ✅ ログイン～アプリケーション画面への遷移が成功
- ✅ 基本的な開発環境の復旧

Phase 2: データ操作エラーの発見（現在地点）

発生事象:

- アカウト一覧ページでCannot read properties of undefined (reading 'list')エラー
- 原因: モック認証環境でAmplifyクライアント（client.models.Account.list()）の実行失敗

暫定対応:

- Accountsページとアカウント詳細ページにモックデータを追加
- 動作確認済み（3つのモックアカウントで正常動作）

判明した課題: 同様のAmplify依存が システム全体に広範囲に存在する可能性

🔍 Amplify依存調査結果

調査範囲

- app/routes/（全ページコンポーネント）
- app/components/（UIコンポーネント）
- app/lib/（ライブラリ・ユーティリティ）
- 設定ファイル・依存関係

依存箇所の分類

🔴 高影響度（8ファイル）

すぐに動作に影響する箇所

ファイル	影響内容	対応難易度
app/root.tsx	Amplify.configure、認証設定	高
app/routes/protected/layout.tsx	fetchUserAttributes、認証チェック	高
app/routes/auth/login.tsx	Authenticator、getCurrentUser	高
app/lib/amplify-client.ts	generateClient（クライアント）	高
app/lib/amplify-ssr-client.ts	generateClient（SSR）	高
app/lib/amplifyServerUtils.ts	runWithAmplifyServerContext	高
app/lib/account.ts	ProjectAssignment CRUD	中
app/lib/project.ts	ProjectTechnologyLink CRUD	中

🟡 中影響度（15ファイル）

データ操作ページ（全CRUD操作）

Accounts関連（4ファイル）：

- `app/routes/accounts.tsx` - 一覧・CSV import
- `app/routes/accounts/$accountId/index.tsx` - 詳細表示
- `app/routes/accounts/$accountId/edit.tsx` - 編集
- `app/routes/accounts/new.tsx` - 新規作成

Projects関連（4ファイル）：

- `app/routes/projects.tsx` - 一覧表示
- `app/routes/projects/$projectId/index.tsx` - 詳細表示
- `app/routes/projects/$projectId/edit.tsx` - 編集
- `app/routes/projects/new.tsx` - 新規作成

Project Technologies関連（4ファイル）：

- `app/routes/project-technologies.tsx` - 一覧表示
- `app/routes/project-technologies/$projectTechnologyId/index.tsx` - 詳細表示
- `app/routes/project-technologies/$projectTechnologyId/edit.tsx` - 編集
- `app/routes/project-technologies/new.tsx` - 新規作成

その他（3ファイル）：

- `app/components/nav-user.tsx` - ログアウト機能
- CSVインポート機能
- フォーム系コンポーネント

● 低影響度（7ファイル）

型定義のみの依存

- `app/components/account-card.tsx`
- `app/components/account-form.tsx`
- `app/components/project-form.tsx`
- その他型定義ファイル

技術的依存関係

package.json依存関係

```
{
  "@aws-amplify/ui-react": "^6.9.1",
  "aws-amplify": "^6.12.3",
  "amplify-adapter-react-router": "^0.1.0",
  "@aws-amplify/backend": "^1.14.0",
  "@aws-amplify/backend-cli": "^1.4.9",
  "vite-plugin-react-router-amplify-hosting": "^0.1.1"
}
```

データモデル

```
// amplify/data/resource.ts
- Account（従業員情報）
- Project（プロジェクト情報）
- ProjectTechnology（技術カタログ）
- ProjectAssignment（アカウント-プロジェクト関連）
- ProjectTechnologyLink（プロジェクト-技術関連）
```

開発方針の選択肢

Option A: 全面モック化アプローチ

実装内容

1. データレイヤーの完全モック化

- 全30ファイルのAmplify呼び出しをモック実装に置換
- インメモリデータストア、または Local Storage / IndexedDB での永続化
- 型定義の独立化

2. 認証システムの拡張

- 既存のモック認証をデータ操作まで拡張
- ユーザー権限、セッション管理の実装

3. 開発ツールの整備

- モックデータの管理システム
- データリセット・シードデータ機能
- デバッグ用データビューアー

メリット

- **完全ローカル開発:** インターネット接続不要
- **高速開発サイクル:** クラウドAPI呼び出しなし
- **AWS費用ゼロ:** 開発段階での課金なし
- **環境構築不要:** 新規開発者のオンボーディングが容易
- **オフライン開発:** ネットワーク環境に依存しない

デメリット

- **大規模実装:** 30ファイル×複数の関数=100+箇所の修正
- **複雑性:** 関連性のあるデータモデルの整合性管理
- **開発コスト:** 2-3週間の実装期間
- **メンテナンス負荷:** モックとプロダクションの2重管理
- **本番差異リスク:** 実環境での予期しない動作
- **機能制限:** AWS固有機能（リアルタイム更新、権限制御等）の再現困難

工数見積もり

- 初期実装: 15-20営業日
- 継続メンテナンス: 月2-3営業日
- 本番移行時の検証: 5-7営業日

Option B: Amplify環境構築アプローチ

実装内容

1. AWS環境のセットアップ

- AWS CLI、Amplify CLIの設定
- **amplify sandbox**を使用した開発環境構築
- 既存スキーマを使用したバックエンドの復元

2. 開発環境の整備

- 実際の**amplify_outputs.json**の取得
- 環境変数の設定
- チーム開発用のサンドボックス共有設定

3. ハイブリッド対応

- 必要に応じて部分的なモック機能の保持
- 環境切り替え機能の拡張

メリット

- **既存資産活用**: 前開発者の設計・実装を100%活用
- **高い整合性**: 本番環境との完全一致
- **短期構築**: 2-3営業日で環境復旧
- **AWS機能フル活用**: リアルタイム、権限制御、分析等
- **将来性**: 本番運用への自然な移行
- **チーム開発**: 複数人での協調開発が容易

デメリット

- **クラウド依存**: インターネット接続必須
- **AWS学習**: Amplify CLI、AWS基礎知識が必要
- **環境管理**: 開発・ステージング・本番の環境管理
- **課金**: 開発環境での最小限の課金発生
- **複雑性**: AWS サービス間の依存関係の理解

工数見積もり

- 初期環境構築: 2-3営業日
- チーム環境整備: 1-2営業日
- 継続メンテナンス: 月1営業日未満

推奨方針

結論: **Option B (Amplify環境構築)** を推奨

推奨理由

1. コストパフォーマンス

- 実装コスト: 3営業日 vs 20営業日（7倍の差）
- 前開発者の投資資産を有効活用

2. 品質・リスク管理

- 本番環境との完全整合性
- AWS専用機能の動作確認
- 予期しない動作の回避

3. 開発体験

- 実際のクラウド環境での開発経験
- AWS エコシステムの学習機会
- 将来的なスケーラビリティの確保

4. 前開発者の意図の尊重

- Amplifyを選択した技術的判断の継承
- 既存アーキテクチャの設計思想の維持

実装段階的アプローチ

Phase 1: 緊急対応（即日～2営業日）

- AWS CLI、Amplify CLIの環境構築
- **amplify sandbox**での最小限開発環境構築
- 既存機能の動作確認

Phase 2: 環境整備（3-5営業日）

- チーム開発環境の設定
- CI/CDパイプラインの構築
- ドキュメント整備

Phase 3: 最適化（継続的）

- パフォーマンス最適化
- セキュリティ設定の強化
- 監視・ログ設定

後退オプション

万が一Amplify環境構築が困難な場合の対応策：

1. ハイブリッドアプローチ: 認証のみAmplify、データ操作はモック
2. 段階的モック化: 高優先度ページから順次モック化
3. 外部API化: JSON Server等でのREST API化

意思決定のための比較表

観点	モック化	Amplify環境	重要度
初期実装コスト	高（20営業日）	低（3営業日）	☆☆☆
学習コスト	低	中	☆☆
開発スピード	高	中	☆☆☆
本番整合性	低	高	☆☆☆
インフラ費用	ゼロ	最小限	☆☆
将来拡張性	低	高	☆☆☆
チーム開発	中	高	☆☆
メンテナンス性	低	高	☆☆☆

総合判定: Amplify環境構築が優位

関係者への確認事項

前開発者への質問

1. 既存環境について

- 使用していたAWSアカウント・リージョン
- Amplify環境の設定・バックアップ状況
- 開発時の環境構築手順やドキュメント

2. 技術選定の背景

- Amplifyを選択した理由・メリット
- 将来的な本番運用の計画
- 推奨する開発フロー

3. 移行支援

- 環境構築のサポート可能性
- 既存設定ファイルの共有
- トラブルシューティングの協力

プロジェクト関係者への確認

1. 予算・リソース

- AWS開発環境の予算承認
- 学習時間の確保
- 外部サポートの必要性

2. スケジュール

- 環境構築の許容期間
- 開発再開の優先度
- リリース予定への影響

3. 方針決定

- 最終的な開発方針の決定権者
- 意思決定のタイムライン
- リスク受容度

次のアクションプラン

即座に必要な決定

1. **開発方針の選択:** モック化 vs Amplify環境
2. **責任者の決定:** 環境構築の実行責任者
3. **タイムライン設定:** 環境構築完了目標日

決定後のアクション

Amplify環境構築を選択した場合

1. 前開発者との技術移行ミーティング設定
2. AWS環境へのアクセス権取得
3. 段階的な環境構築の実行

モック化を選択した場合

1. 詳細実装計画の策定
2. モックデータ設計の仕様決定
3. 段階的実装のマイルストーン設定

補足情報

現在の対応状況

-  モック認証: 実装完了・テスト済み
-  Accountsページ: 暫定的にモック対応（コミット保留中）
-  Projects、ProjectTechnologies: 未対応（同様のエラーが予想される）

緊急性

- 開発継続のため、1週間以内の方針決定が望ましい

- 現在は限定的な機能（認証周り）のみ動作可能

技術的補足

本レポートに記載されていない技術的詳細については、以下のドキュメントを参照：

- [docs/logs/20250705_モック認証システム実装作業報告.md](#)
- [docs/logs/20250705_PlaywrightE2E検証環境構築作業報告.md](#)
- [docs/development/testing-guide.md](#)

レポート終了

本レポートを基に、関係者間での協議と最終的な開発方針の決定をお願いいたします。決定され次第、速やかに実装に着手いたします。